Take-home assignment: “AI Incident Assistant” (up to 3 hours)

The goal is to evaluate your ability to design and implement a small AI agent that uses an LLM as a controlled component in a service architecture, not as a single black-box function.

Estimated time limit: **up to 3 hours** of focused work. Clean design and reasonable trade-offs are more important than exhaustive edge-case coverage.

Context

Imagine you are building an internal tool for on-call engineers to help them triage production incidents.

You are given:

1. **System description (simplified)**

```
Our payment platform consists of the following main services:

- api-gateway: receives external HTTP requests from clients and routes them to internal services.
- auth-service: authentication and token issuance.
- payment-service: creation and processing of payment transactions.
- billing-service: balance management and invoicing.
- notification-service: sending e-mail and SMS notifications.
- reporting-service: generating reports and exporting data.

General notes:
- All services write logs to a centralized log storage (ELK).
- The main database is PostgreSQL, with separate instances for payment-service and billing-service.
- Payments often experience external provider errors (timeout, 5xx, invalid credentials).
- notification-service may degrade when external SMTP/SMS providers have issues.
- reporting-service puts extra load on the DB with long analytical queries.
```

2. **Examples of past incidents**

```
[INC-101] Customers report that they cannot pay by card.
In payment-service logs we see massive timeouts when calling the PayGate provider.
The issue started around 12:05 UTC.
No other metric anomalies were noticed.
---

[INC-102] Sharp increase in response time for /payments/create (up to 5-7 seconds).
DB dashboards show high CPU and many long-running queries from reporting-service.
Some customers receive 504 Gateway Timeout from api-gateway.
---

[INC-103] Users do not receive top-up confirmation e-mails.
Money is credited successfully, balances are correct.
In notification-service logs there are intermittent connection errors to the SMTP provider.
---

[INC-104] Some customers cannot log in via the mobile app.
auth-service responds with 401 errors.
Logs show messages about invalid token signatures.
No large-scale failures in other services.
```

Your task

Implement a small service (CLI tool, HTTP API or simple web UI — your choice) that, given an incident description text:

1. **Classifies the incident**, for example:
   - “External payment provider issue”
   - “DB degradation caused by reporting”
   - “Notification delivery issue”
   - “User authentication errors”
2. **Produces a short summary**:
   - what is happening;
   - who is likely affected;
   - how critical it is (low/medium/high — you decide on the scale).
3. **Suggests hypotheses and diagnostic steps**:
   - up to 3 hypotheses about possible root causes;
   - for each hypothesis, 2–3 concrete next steps (what to check, which logs/metrics to look at, etc.).

LLM/agent requirements

Mandatory points:

- Use the LLM **as part of an agent architecture**, not as a single “magic” function:
  - Explicitly separate stages: parsing the input → using auxiliary data (system description, past incidents) → generating a structured answer.
  - This can be implemented as a custom orchestrator or via any agent framework.
- Return the result in a **machine-readable format**:
  - For example, JSON with the following (or similar) structure:

```
{
  "category": "string",
  "summary": "string",
  "severity": "low|medium|high",
  "hypotheses": [
    {
      "title": "string",
      "reasoning": "string",
      "next_steps": ["string", "string"]
    }
  ]
}
```

- Handle typical model errors:
  - invalid JSON;
  - unexpected output format;
  - response in a language you do not expect, etc.
- Show a recovery strategy:
  - for example, a retry with a refined prompt;
  - validation against a JSON schema with auto-correction / regeneration.

Technical conditions

- Programming language and stack — your choice.
- You may use:
  - any public LLM API (OpenAI, Anthropic, Gemini, etc.) or a local model;
  - any LLM/agent libraries/frameworks (LangChain, Semantic Kernel, etc.).
- Important:
  - Move configuration (API keys, endpoints) to external settings/environment variables.
  - Keep the solution compact and readable — no need for heavy enterprise boilerplate.

What to submit

1. A link to the repository (GitHub/GitLab/Bitbucket, etc.) with your code.
2. A **README** briefly describing:
   - how to run the project (minimal steps);
   - how the agent is structured at a high level (1–2 paragraphs or a simple step-by-step description);
   - how you tested the behavior (e.g., a list of 3–5 test incidents and what kind of outputs you expected).
3. (Optional, but appreciated) a short note on trade-offs:
   - what you simplified to fit into ~3 hours;
   - what you would do differently with more time.

Mini example of expected behavior (non-canonical)

Input:

```
 Customers complain that card payments often fail, and transactions do not go through.
payment-service logs show many timeouts when calling PayGate, starting from 12:05 UTC.
Other services look normal.
```

Example of a possible (schematic) answer:

```
{
  "category": "External payment provider issue",
  "summary": "The external provider PayGate is not responding in time, causing mass card payment failures.",
  "severity": "high",
  "hypotheses": [
    {
      "title": "Degradation or incident on the PayGate side",
      "reasoning": "Timeouts are observed only when calling PayGate, other services remain stable.",
      "next_steps": [
        "Check PayGate status page and recent provider notifications.",
        "Compare error and latency metrics for PayGate vs other payment providers.",
        "If possible, temporarily shift part of the traffic to an alternative provider."
      ]
    }
  ]
}
```